That takes care of building pipelines and registering services conditionally based on the current environment, but what about other situations? Suppose you only want to log informational messages in development or you want to fetch external resources from a content delivery network (CDN) only in production. Depending on the situation, you can access the `IHostEnvironment` service to establish the current environment, or you can use the environment tag helper in Razor pages.

14.1.4 The IHostEnvironment service

The `IHostEnvironment` service provides information about the application's hosting environment. You have already encountered this type on a couple of occasions previously. As you have just seen, it is made available via the `WebApplication` and the `WebApplicationBuilder` `Environment` property. You used it within your original `EmailService` in chapter 9 too, when you used its `ContentRootPath` property to build a path to the email pickup folder. The `ContentRootPath` is the absolute path to the application's root directory. There is also a very similar `IWebHostEnvironment` service. This implements `IHostEnvironment`, providing access to `Environment` and `ContentRootPath`, but also exposes `WebRootPath`, which enables you to get or set the path to the content or static files—wwwroot by default.

Returning to the scenario in which you only want to log informational messages in development, you only need access to the `Environment`, so you will use `IHostEnvironment` to get this information. You'll alter the existing `IndexModel` to inject the `IHostEnvironment` services as a dependency and assign `IHostEnvironment` to a private